

Summary

Describe your unit testing approach for each of the three features.

My approach for each feature was the same. I would implement the features by following the descriptions in the requirements, then build the tests that would ensure each function kept their variables within the requirement descriptions, after this I would run the tests then make any changes needed to the code if any tests failed, then repeat.

All my tests were reflections of the requirements set and my code was made to meet all the requirements while adding extra functions only when necessary, such as the hash and equal functions to allow classes to be used in maps. The tests are obvious reflections as they test to make sure variables stay in the required limits, such as character length and not being null.

The tests were effective as they cover most of the code. All class functions were tested using the requirements except in cases where there would be overlap in the tests performed, such as the constructors that used set methods. Even though set methods were used in the constructor tests were still performed on variables that were not set with set methods, such as the immutable variables. This was done so that all variable changes were thoroughly tested but testing redundancy was reduced.

Describe your experience writing the JUnit tests.

I ensured my code was technically sound by utilizing proper data structures, properly scoped functions, and data validation.

For example all of the service classes were containers for an object class, such as the Appointment Service class containing the Appointment class. Since all of these containers were to contain unique objects that had IDs I utilized the HashMap data structure and used the IDs of the objects as keys to enforce uniqueness so that no duplicates could be added. The IDs were kept unique by using the “final” keyword to make sure the ID could not be changed. Objects added to the services were then validated using their IDs to make sure the ID was not already in the container before adding them. Similarly all setter functions validate the argument input before setting the variables to ensure they stay within requirements.

I also kept functions simple and to their main purpose in order to follow coding standards. An example is in the Task Service class line 35 where I created a validation function for IDs since the validation was used throughout multiple other functions where IDs were input as arguments.

My code was efficient due to the same reasons above, the HashMap data structure allowed for fast and efficient storage while functions kept to their scope would prevent redundancy, but additionally I made sure to not have redundant tests to prevent unnecessary double checking and keeping the tests fast. The best example of this is the Task Service Tests. One of the functionalities of the Task Service was to be able to update Tasks by using the ID of a task and passing the new value for a variable such as the description or name. The tests for the Task Service were kept efficient by only testing to make sure IDs passed to the update functions were valid since the functions used the setter functions on the Task class which would be tested by the Task Tests. These setter functions would already handle validation of input and thus not need to be tested again by the Task Service Tests.

Reflection

Testing Techniques

The project uses Unit Testing by utilizing JUnit to perform the unit tests. Unit tests involve testing the code, section by section, in “units” (usually the functions of a class). This is to ensure the functionality of classes work on their own when provided their expected ranges of input, as well as the function’s ability to handle bad input such as null or out of range inputs.

Unit tests are a form of “white box” testing since they are designed to test sections of the code with a focus on how they work, rather than testing the overall functionality of the code. They are written to test specific functions within an isolated test environment so that all variables are known and only the output of the single function is the focus of an individual test, making it very necessary for there to be many tests to cover all the functions and their respective testing scenarios, but this also makes unit tests very thorough.

There are many testing techniques that were not used such as integration testing. Integration testing focuses on the interactions between code modules to make sure they are working together properly. No security testing was used either to detect any security vulnerabilities through the use of penetration testing or security audits. Performance tests were also not used such as load testing to which simulates many users using the system, or soak testing which detects memory leaks. None of these were used as the code is still too early and that was not the focus of these milestones.

Unit tests are good for finding errors in functions and pinpointing the exact function that has the error, where any black box test would not know the source of the issue. Unit tests also

allow for projects to make sure requirements are maintained as the project grows by utilizing regression which will ensure old tests are still performed on the code as additional tests and requirements are added. The nature of unit tests will also reduce debugging time as error location is found faster and improve code quality as the tests help ensure developers do not submit poor performing code.

Integration testing is good for when a project is bringing sections of code together. This testing will help developers make sure the sections of the code are properly working together and will help developers find any issues with data exchanges between the sections.

Security testing is important for projects at most points in development and will help a project prevent unrestricted or unauthorized access to parts of the code or data. Tests like penetration tests are used to simulate an attacker trying to access functionality in the code they would not normally have access to through the use of several techniques that may exploit weaknesses in the design of the system such as bad input handling or improper authorization of users.

Mindset

While working on the project my mindset was relatively the same as it is when working on other coding projects, however with the addition of having to write and perform unit tests I found myself wondering how the code would perform in the unit tests. In addition to my usual problem solving mindset I was not solely focused on trying to implement the code following requirements, I had to think about the unit tests as well and how to make the tests. This was like having to think twice about what I was making, almost as if the testing was another layer of requirements I had to ensure my code met.

To limit bias I tried to make sure to test everything I could to ensure the code held up to the requirements, even if I was sure they would pass the tests. I adopted the mindset that the unit tests were concrete proof of the code being ready, even though I had written the code myself and knew it met requirements. This helped me ensure I wrote tests for even simple setter functions with straight forward input validation. This also helped to prevent bias toward my own code and assuming it would work.

Being disciplined to provide quality code is important as a software engineer because it is the job and that is the skill you are being paid for, otherwise someone else could take your place. This is also why corners should not be cut since it can degrade the quality of the code and that would impact the functionality of the product which could create a domino effect that results in the company or team's professionalism and ability being put into question. As I touched on while talking about bias, if I had skipped writing some tests or all of them because I assumed they would work then bugs could easily find their way into production. Additionally testing could take longer to solve a problem if the tests were not used and only a code review was used.

Technical debt seems like a large topic and something that is easier said than done when it comes to preventing it but by taking the time to properly plan and implement systems while doing proper testing, technical debt can be reduced. Setting up tests will even help manage tech debt by preventing bugs from being introduced later on in production that may change how already good systems perform since the tests can be run on any changed code once the tests are made.